

Document number: P0433R3

Date: 2017-06-18

Reply-To:

Mike Spertus, Symantec (mike_spertus@symantec.com)

Walter E. Brown (webrown.cpp@gmail.com)

Stephan T. Lavavej (stl@exchange.microsoft.com)

Audience: {Library Evolution, Library} Working Group

Toward a resolution of US7 and US14: Integrating template deduction for class templates into the standard library

Introduction

National body comments US7 and US14 request analysis of the standard library to determine what changes might be desirable in light of the C++17 adoption of [P0091R3](#) (Template argument deduction for class templates (rev. 6)). The [previous revision](#) of this paper describes the changes that were adopted into the C++17 standard libraries. In this paper, we describe some additional changes that we believe could improve the feature hope may be considered for adoption as a DR.

Note that, in addition to the proposals in this paper, there are two LWG issues relating to deduction guides in the standard library that should be addressed in LWG issue processing:

1. [2981. Remove redundant deduction guides from standard library](#)
2. [2982. Making size type consistent in associative container deduction guides](#)

scoped_lock argument order

As Jonathan Wakely has noted, code like the following does not work:

```
mutex m;  
scoped_lock l(m, adopt_lock);
```

The point here is that when the compiler attempts to deduce `scoped_lock<m1, adopt_lock>`, there is a hard error outside the immediate context as `adopt_lock` is not lockable. Note that the same code works fine if `scoped_lock` is replaced by `lock_guard` because `lock_guard` does not have a variadic argument.

We recommend fixing this by moving the `adopt_lock_t` parameter to the front of the parameter list. This was not done originally because `scoped_lock` was originally named `lock_guard` and had to remain compatible to avoid code breakage, which is no longer a requirement now that `scoped_lock` is a different class than `lock_guard`. This is useful to fix because it will encourage programmers to migrate to consistent use of `scoped_lock` rather than choosing one or the other based on circumstances.

Wording

In the definition of `class scoped_lock` in §33.4.4.2 [thread.lock.scoped], make the following change:

```
explicit scoped_lock(MutexTypes&... m);  
explicit scoped_lock(MutexTypes&... m, adopt_lock_t);  
explicit scoped_lock(adopt_lock_t, MutexTypes&... m);  
~scoped_lock();
```

Likewise, change §33.4.4.2 [thread.lock.scoped] starting immediately before paragraph 4 as follows:

```
explicit scoped_lock(MutexTypes&... m, adopt_lock_t);  
explicit scoped_lock(adopt_lock_t, MutexTypes&... m);
```

Requires: The calling thread owns all the mutexes in `m`.

Enable variant support

The following code fails to compile

```
variant<int, double> v1(3);  
variant v2 = v1; // ill-formed!
```

As this natural code is useful and its failure is confusing, we propose that it be supported. Indeed, prior to the adoption of p0510r0 banning `variant<>`, the above code worked as expected since `variant<>` occurs in some deduction guides in the overload

set. As it is not clear that constructor template argument deduction was considered in adopting p0510r0, we would like to consider allowing `variant<>` not to produce a hard error in such cases.

Restoring missing container deduction guides

A number of associative container guides were removed in the late stages of the Kona meeting when it was realized that the `size_type` member may or may not be a deduced context, and different compilers handle “most specialized” differently when some candidates are deducible and others are non-deducible for the same argument (expect a paper on this in the future!). In addition, construction from allocators was removed from all other containers for consistency.

Using `size_t` is a nice way to do this (although note that the original concern may have been an hallucination to begin with), allowing the full set of deduction guides to be restored. Note that using `size_t` does not assume that the container's `size_type` is `size_t`, since it is just an integral type being used for overload resolution. We adopt this in the wording below, which also simplifies and strengthens the existing language. While one could reasonably argue that constructing from a container is not the most important use case, perhaps a better argument is that consistently providing deduction guides to support all of the STL container constructors that make sense for deduction rather than picking and choosing is less surprising and artificial.

Wording

While this wording appears rather long, note that it mainly consists of undoing the removal of deduction guides that were apparently uncontroversial in prior versions of this paper until the above, now addressed, concern arose late in the Kona meeting. The only other change is the consistent adoption of `size_t`, which is a simplification (that also increases robustness)

Add the following deduction guide to the definition of class `basic_string` in §24.3.2 [basic.string]:

```
int compare(size_type pos1, size_type n1,
            const charT* s, size_type n2) const;
};
```

```
template<typename Allocator>
basic_string(Allocator) -> basic_string<typename Allocator::value_type,
char_traits<typename Allocator::value_type>, Allocator>;
```

In §24.3.2.2 [string.cons], insert the following:

```
template<typename Allocator>
basic_string(Allocator) -> basic_string<typename Allocator::value_type,
char_traits<typename Allocator::value_type>, Allocator>;
```

Remarks: Shall not participate in overload resolution if `Allocator` is called with a type that does not qualify as an allocator [sequence.reqmts].

At the end of the definition of class `deque` in §26.3.8.1 [deque.overview], add the following deduction guides:

```
void clear() noexcept;
};

template<class Allocator>
deque(Allocator) -> deque<typename Allocator::value_type, Allocator>;

template<class Allocator>
deque(size_t, Allocator) -> deque<typename Allocator::value_type, Allocator>;
```

At the end of the definition of class `forward_list` in §26.3.9.1 [forwardlist.overview], add the following deduction guides:

```
void reverse() noexcept;
};

template<class Allocator>
forward_list(Allocator) -> forward_list<typename Allocator::value_type, Allocator>;

template<class Allocator>
forward_list(size_t, Allocator) -> forward_list<typename Allocator::value_type, Allocator>;
```

At the end of the definition of class `list` in §26.3.10.1 [list.overview], add the following deduction guides:

```
void reverse() noexcept;
};

template<class Allocator>
list(Allocator) -> list<typename Allocator::value_type, Allocator>;
```

```
template<class Allocator>
list(size_t, Allocator) -> list<typename Allocator::value_type, Allocator>;
```

At the end of the definition of class `vector` in §26.3.11.1 [vector.overview], add the following *deduction-guide*:

```
void clear() noexcept;
};

template<class Allocator>
vector(Allocator) -> vector<typename Allocator::value_type, Allocator>;

template<class Allocator>
vector(size_t, Allocator)
-> vector<typename Allocator::value_type, Allocator>;
```

Add the following to the end of §26.4.1p2 [associative.general]

```
template<class InputIterator>
using iter_to_alloc_t = pair<add_const_t<typename iterator_traits<InputIterator>::value_type::first_type>,
                           typename iterator_traits<InputIterator>::value_type::second_type> // exposition only

template<class Allocator>
using alloc_key_t = remove_const_t<typename Allocator::value_type::first_type>; // exposition only
template<class Allocator>
using alloc_val_t = typename Allocator::value_type::second_type; // exposition only
```

At the end of the definition of class `map` in §26.4.4.1 [map.overview], add the following deduction guides:

```
pair<const_iterator, const_iterator> equal_range(const K& x) const;
};

template<class Compare, class Allocator>
map(Compare, Allocator)
-> map<alloc_key_t<Allocator>, alloc_val_t<Allocator>, Compare, Allocator>;

template <class InputIterator,
          class Compare = less<iter_key_t<InputIterator>>,
          class Allocator = allocator<iter_to_alloc_t<InputIterator>>>
map(InputIterator, InputIterator, Compare = Compare(), Allocator = Allocator())
-> map<iter_key_t<InputIterator>, iter_val_t<InputIterator>, Compare, Allocator>;

template<class Allocator>
map(Allocator)
-> map<alloc_key_t<Allocator>, alloc_val_t<Allocator>,
      less<alloc_key_t<Allocator>>, Allocator>;
```

At the end of the definition of class `multimap` in §26.4.4.1 [multimap.overview], add the following deduction guides:

```
pair<const_iterator, const_iterator> equal_range(const K& x) const;
};

template<class Compare, class Allocator>
multimap(Compare, Allocator)
-> multimap<alloc_key_t<Allocator>, alloc_val_t<Allocator>, Compare, Allocator>;

template <class InputIterator,
          class Compare = less<iter_key_t<InputIterator>>,
          class Allocator = allocator<iter_to_alloc_t<InputIterator>>>
multimap(InputIterator, InputIterator, Compare = Compare(), Allocator = Allocator())
-> multimap<iter_key_t<InputIterator>, iter_val_t<InputIterator>, Compare, Allocator>;

template<class Allocator>
multimap(Allocator)
-> multimap<alloc_key_t<Allocator>, alloc_val_t<Allocator>,
      less<alloc_key_t<Allocator>>, Allocator>;
```

At the end of the definition of class `set` in §26.4.6.1 [set.overview], add the following *deduction-guides*:

```
template <class K>
pair<const_iterator, const_iterator> equal_range(const K& x) const;
};

template<class Compare, class Allocator>
set(Compare, Allocator)
-> set<typename Allocator::value_type, Compare, Allocator>;

template <class InputIterator,
          class Compare = less<typename iterator_traits<InputIterator>::value_type>,
          class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
```

```

set(InputIterator, InputIterator,
    Compare = Compare(), Allocator = Allocator())
-> set<typename iterator_traits<InputIterator>::value_type, Compare, Allocator>;

```

```

template<class Allocator>
set(Allocator)
-> set<typename Allocator::value_type, less<typename Allocator::value_type>, Allocator>;

```

At the end of the definition of class `multiset` in §27.4.6.1 [multiset.overview], add the following deduction guides:

```

template <class K>
pair<const_iterator, const_iterator> equal_range(const K& x) const;
};

```

```

template<class Compare, class Allocator>
multiset(Compare, Allocator)
-> multiset<typename Allocator::value_type, Compare, Allocator>;

```

```

template <class InputIterator,
    class Compare = less<typename iterator_traits<InputIterator>::value_type>,
    class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
multiset(InputIterator, InputIterator,
    Compare = Compare(), Allocator = Allocator())
-> multiset<typename iterator_traits<InputIterator>::value_type, Compare, Allocator>;

```

```

template<class Allocator>
multiset(Allocator)
-> multiset<typename Allocator::value_type, less<typename Allocator::value_type>, Allocator>;

```

Modify §26.5.1p2 [unord.general] as follows:

The exposition-only alias templates `iter_key_t`, `iter_val_t`, `alloc_key_t`, `alloc_val_t`, and `iter_to_alloc_t` defined in 26.4.1 may appear in deduction guides for unordered containers.

Modify §26.5.4.1 [unord.map.overview], add the following *deduction-guides*:

```

void reserve(size_type n);
};

```

```

template<class Hash, class Pred, class Allocator>
unordered_map(size_t, Hash, Pred, Allocator)
-> unordered_map<alloc_key_t<Allocator>, alloc_val_t<Allocator>,
    Hash, Pred, Allocator>;

```

```

template<class InputIterator,
    class Hash = hash<iter_key_t<InputIterator>>, class Pred = equal_to<iter_key_t<InputIterator>>,
    class Allocator = allocator<iter_to_alloc_t<InputIterator>>>
unordered_map(InputIterator, InputIterator, typename see below::size_typesize_t = see below,
    Hash = Hash(), Pred = Pred(), Allocator = Allocator())
-> unordered_map<iter_key_t<InputIterator>, iter_value_t<InputIterator>, Hash, Pred, Allocator>;

```

```

template<class Allocator>
unordered_map(Allocator)
-> unordered_map<alloc_key_t<Allocator>, alloc_val_t<Allocator>,
    hash<alloc_key_t<Allocator>>, equal_to<alloc_key_t<Allocator>>, Allocator>;

```

```

template<class Key, class T, class Hash = hash<Key>,
    class Pred = equal_to<Key>, class Allocator = allocator<pair<const Key, T>>>
unordered_map(initializer_list<pair<const Key, T>>, typename see below::size_typesize_t = see below,
    Hash = Hash(), Pred = Pred(), Allocator = Allocator())
-> unordered_map<Key, T, Hash, Pred, Allocator>;

```

```

template<class Allocator>
unordered_map(size_t, Allocator)
-> unordered_map<alloc_key_t<Allocator>, alloc_val_t<Allocator>,
    hash<alloc_key_t<Allocator>>, equal_to<alloc_key_t<Allocator>>, Allocator>;

```

```

template<class Hash, class Allocator>
unordered_map(size_t, Hash, Allocator)
-> unordered_map<alloc_key_t<Allocator>, alloc_val_t<Allocator>,
    Hash, equal_to<alloc_key_t<Allocator>>, Allocator>;

```

```

template<class InputIterator, class Allocator>
unordered_map(InputIterator, InputIterator, typename see below::size_typesize_t, Allocator)
-> unordered_map<iter_key_t<InputIterator>, iter_val_t<InputIterator>,
    hash<iter_key_t<InputIterator>>, equal_to<iter_key_t<InputIterator>>, Allocator>;

```

```

template<class InputIterator, class Allocator>

```

```

unordered_map(InputIterator, InputIterator, Allocator)
-> unordered_map<iter_key_t<InputIterator>, iter_val_t<InputIterator>,
    hash<iter_key_t<InputIterator>>, equal_to<iter_key_t<InputIterator>>, Allocator>;

template<class InputIterator, class Hash, class Allocator>
unordered_map(InputIterator, InputIterator, typename see_below::size_type size_t, Hash, Allocator)
-> unordered_map<iter_key_t<InputIterator>, iter_val_t<InputIterator>, Hash,
    equal_to<iter_key_t<InputIterator>>, Allocator>;

template<class Key, class T, typename Allocator>
unordered_map(initializer_list<pair<const Key, T>>, typename see_below::size_type size_t, Allocator)
-> unordered_map<Key, T, hash<Key>, equal_to<Key>, Allocator>;

template<class Key, class T, typename Allocator>
unordered_map(initializer_list<pair<const Key, T>>, Allocator)
-> unordered_map<Key, T, hash<Key>, equal_to<Key>, Allocator>;

template<class Key, class T, class Hash, class Allocator>
unordered_map(initializer_list<pair<const Key, T>>, typename see_below::size_type size_t, Hash, Allocator)
-> unordered_map<Key, T, Hash, equal_to<Key>, Allocator>;

```

Delete §26.5.4.1p4 [unord.map.overview]:

A `size_type` parameter type in an `unordered_map` deduction guide refers to the `size_type` member type of the type deduced by the deduction guide.

Modify §26.5.5.1 [unord.map.overview], add the following *deduction-guides*:

```

void reserve(size_type n);
};

template<class Hash, class Pred, class Allocator>
unordered_multimap(size_t, Hash, Pred, Allocator)
-> unordered_multimap<alloc_key_t<Allocator>, alloc_val_t<Allocator>,
    Hash, Pred, Allocator>;

template<class InputIterator,
    class Hash = hash<iter_key_t<InputIterator>>, class Pred = equal_to<iter_key_t<InputIterator>>,
    class Allocator = allocator<iter_to_alloc_t<InputIterator>>>
unordered_multimap(InputIterator, InputIterator, typename see_below::size_type size_t = see below,
    Hash = Hash(), Pred = Pred(), Allocator = Allocator())
-> unordered_multimap<iter_key_t<InputIterator>, iter_value_t<InputIterator>, Hash, Pred, Allocator>;

template<class Allocator>
unordered_multimap(Allocator)
-> unordered_multimap<alloc_key_t<Allocator>, alloc_val_t<Allocator>,
    hash<alloc_key_t<Allocator>>, equal_to<alloc_key_t<Allocator>>, Allocator>;

template<class Key, class T, class Hash = hash<Key>,
    class Pred = equal_to<Key>, class Allocator = allocator<pair<const Key, T>>>
unordered_multimap(initializer_list<pair<const Key, T>>, typename see_below::size_type size_t = see below,
    Hash = Hash(), Pred = Pred(), Allocator = Allocator())
-> unordered_multimap<Key, T, Hash, Pred, Allocator>;

template<class Allocator>
unordered_multimap(size_t, Allocator)
-> unordered_multimap<alloc_key_t<Allocator>, alloc_val_t<Allocator>,
    hash<alloc_key_t<Allocator>>, equal_to<alloc_key_t<Allocator>>, Allocator>;

template<class Hash, class Allocator>
unordered_multimap(size_t, Hash, Allocator)
-> unordered_multimap<alloc_key_t<Allocator>, alloc_val_t<Allocator>,
    Hash, equal_to<alloc_key_t<Allocator>>, Allocator>;

template<class InputIterator, class Allocator>
unordered_multimap(InputIterator, InputIterator, typename see_below::size_type size_t, Allocator)
-> unordered_multimap<iter_key_t<InputIterator>, iter_val_t<InputIterator>,
    hash<iter_key_t<InputIterator>>, equal_to<iter_key_t<InputIterator>>, Allocator>;

template<class InputIterator, class Allocator>
unordered_multimap(InputIterator, InputIterator, Allocator)
-> unordered_multimap<iter_key_t<InputIterator>, iter_val_t<InputIterator>,
    hash<iter_key_t<InputIterator>>, equal_to<iter_key_t<InputIterator>>, Allocator>;

template<class InputIterator, class Hash, class Allocator>
unordered_multimap(InputIterator, InputIterator, typename see_below::size_type size_t, Hash, Allocator)
-> unordered_multimap<iter_key_t<InputIterator>, iter_val_t<InputIterator>, Hash,
    equal_to<iter_key_t<InputIterator>>, Allocator>;

```

```

template<class Key, class T, typename Allocator>
unordered_multimap(initializer_list<pair<const Key, T>>, typename see below::size_typesize_t, Allocator)
    -> unordered_multimap<Key, T, hash<Key>, equal_to<Key>, Allocator>;

template<class Key, class T, typename Allocator>
unordered_multimap(initializer_list<pair<const Key, T>>, Allocator)
    -> unordered_multimap<Key, T, hash<Key>, equal_to<Key>, Allocator>;

template<class Key, class T, class Hash, class Allocator>
unordered_multimap(initializer_list<pair<const Key, T>>, typename see below::size_typesize_t, Hash, Allocator)
    -> unordered_multimap<Key, T, Hash, equal_to<Key>, Allocator>;

```

Delete §26.5.5.1p4 [unord.multimap.overview]:

A ~~size_type~~ parameter type in an unordered_multimap deduction guide refers to the ~~size_type~~ member type of the type deduced by the deduction guide.

Modify §26.5.6.1 [unord.set.overview] as follows:

```

void reserve(size_type n);
};

template<class Hash, class Pred, class Allocator>
unordered_set(size_t, Hash, Pred, Allocator)
    -> unordered_set<typename Allocator::value_type, Hash, Pred, Allocator>;

template<class InputIterator,
        class Hash = hash<typename iterator_traits<InputIterator>::value_type>,
        class Pred = equal_to<typename iterator_traits<InputIterator>::value_type>,
        class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
unordered_set(InputIterator, InputIterator, typename see below::size_typesize_t = see below,
        Hash = Hash(), Pred = Pred(), Allocator = Allocator())
    -> unordered_set<typename iterator_traits<InputIterator>::value_type,
        Hash, Pred, Allocator>;

template<class T, class Hash = hash<T>,
        class Pred = equal_to<T>, class Allocator = allocator<T>>
unordered_set(initializer_list<T>, typename see below::size_typesize_t = see below,
        Hash = Hash(), Pred = Pred(), Allocator = Allocator())
    -> unordered_set<T, Hash, Pred, Allocator>;

template<class Allocator>
unordered_set(typename see below::size_typesize_t, Allocator)
    -> unordered_set<typename Allocator::value_type, hash<typename Allocator::value_type>,
        equal_to<typename Allocator::value_type>, Allocator>;

template<class Hash, class Allocator>
unordered_set(typename see below::size_typesize_t, Hash, Allocator)
    -> unordered_set<typename Allocator::value_type, Hash,
        equal_to<typename Allocator::value_type>, Allocator>;

template<class InputIterator, class Allocator>
unordered_set(InputIterator, InputIterator, typename see below::size_typesize_t, Allocator)
    -> unordered_set<typename iterator_traits<InputIterator>::value_type,
        hash<typename iterator_traits<InputIterator>::value_type>,
        equal_to<typename iterator_traits<InputIterator>::value_type>,
        Allocator>;

template<class InputIterator, class Hash, class Allocator>
unordered_set(InputIterator, InputIterator, typename see below::size_typesize_t,
        Hash, Allocator)
    -> unordered_set<typename iterator_traits<InputIterator>::value_type, Hash,
        equal_to<typename iterator_traits<InputIterator>::value_type>,
        Allocator>;

template<class T, class Allocator>
unordered_set(initializer_list<T>, typename see below::size_typesize_t, Allocator)
    -> unordered_set<T, hash<T>, equal_to<T>, Allocator>;

template<class T, class Hash, class Allocator>
unordered_set(initializer_list<T>, typename see below::size_typesize_t, Hash, Allocator)
    -> unordered_set<T, Hash, equal_to<T>, Allocator>;

```

Delete §26.5.6.1p4 [unord.set.overview]:

A ~~size_type~~ parameter type in an unordered_set deduction guide refers to the ~~size_type~~ member type of the primary

~~unordered_set template.~~

xxx Modify §26.5.7.1 [unord.multiset.overview] as follows:

```
void reserve(size_type n);
};

template<class Hash, class Pred, class Allocator>
unordered_multiset(size_t, Hash, Pred, Allocator)
-> unordered_multiset<typename Allocator::value_type, Hash, Pred, Allocator>;

template<class InputIterator,
        class Hash = hash<typename iterator_traits<InputIterator>::value_type>,
        class Pred = equal_to<typename iterator_traits<InputIterator>::value_type>,
        class Allocator = allocator<typename iterator_traits<InputIterator>::value_type>>
unordered_multiset(InputIterator, InputIterator, typename see below::size_type size_t = see below,
                  Hash = Hash(), Pred = Pred(), Allocator = Allocator())
-> unordered_multiset<typename iterator_traits<InputIterator>::value_type,
                    Hash, Pred, Allocator>;

template<class T, class Hash = hash<T>,
        class Pred = equal_to<T>, class Allocator = allocator<T>>
unordered_multiset(initializer_list<T>, typename see below::size_type size_t = see below,
                  Hash = Hash(), Pred = Pred(), Allocator = Allocator())
-> unordered_multiset<T, Hash, Pred, Allocator>;

template<class Allocator>
unordered_multiset(typename see below::size_type size_t, Allocator)
-> unordered_multiset<typename Allocator::value_type, hash<typename Allocator::value_type>,
                    equal_to<typename Allocator::value_type>, Allocator>;

template<class Hash, class Allocator>
unordered_multiset(typename see below::size_type size_t, Hash, Allocator)
-> unordered_multiset<typename Allocator::value_type, Hash,
                    equal_to<typename Allocator::value_type>, Allocator>;

template<class InputIterator, class Allocator>
unordered_multiset(InputIterator, InputIterator, typename see below::size_type size_t, Allocator)
-> unordered_multiset<typename iterator_traits<InputIterator>::value_type,
                    hash<typename iterator_traits<InputIterator>::value_type>,
                    equal_to<typename iterator_traits<InputIterator>::value_type>,
                    Allocator>;

template<class InputIterator, class Hash, class Allocator>
unordered_multiset(InputIterator, InputIterator, typename see below::size_type size_t,
                  Hash, Allocator)
-> unordered_multiset<typename iterator_traits<InputIterator>::value_type, Hash,
                    equal_to<typename iterator_traits<InputIterator>::value_type>,
                    Allocator>;

template<class T, class Allocator>
unordered_multiset(initializer_list<T>, typename see below::size_type size_t, Allocator)
-> unordered_multiset<T, hash<T>, equal_to<T>, Allocator>;

template<class T, class Hash, class Allocator>
unordered_multiset(initializer_list<T>, typename see below::size_type size_t, Hash, Allocator)
-> unordered_multiset<T, Hash, equal_to<T>, Allocator>;
```

Delete §26.5.7.1p4 [unord.multiset.overview]:

~~A size_type parameter type in an unordered_multiset deduction guide refers to the size_type member type of the primary unordered_multiset template.~~

At the end of the definition of class queue in §26.6.4.1 [queue.defn], add the following deduction guide

```
void swap(queue& q) noexcept(is_nothrow_swappable_v<Container>)
{ using std::swap; swap(c, q.c); }
};
```

```
template<class Allocator>
queue(Allocator) -> queue<typename Allocator::value_type>;
```

At the end of the definition of class priority_queue in §26.6.6.5 [priority.queue], add the following deduction guides:

```
void swap(priority_queue& q) noexcept(is_nothrow_swappable_v<Container> &&
                                     is_nothrow_swappable_v<Compare>)
{ using std::swap; swap(c, q.c); swap(comp, q.comp); }
```



```
};
```

```
template<class Allocator>
priority_queue(Allocator) -> priority_queue<typename Allocator::value_type>;

template<class Compare, class Allocator>
priority_queue(Compare, Allocator)
-> priority_queue<typename Allocator::value_type, vector<typename Allocator::value_type>, Compare>;
```

At the end of the definition of class `stack` in §26.6.6.1 [`stack`], add the following deduction guide:

```
void swap(stack& q) noexcept(is_nothrow_swappable_v<Container>)
{ using std::swap; swap(c, q.c); }
};

template<class Allocator>
stack(Allocator) -> stack<typename Allocator::value_type>;
```

At the end of the definition of class `promise` in §33.6.6 [`futures.promise`], insert the following:

```
// setting the result with deferred notification
void set_value_at_thread_exit(see below);
void set_exception_at_thread_exit(exception_ptr p);
};
```

```
template <class Alloc> promise(allocator_arg_t, Alloc)
-> promise<typename Alloc::value_type>;
```

```
template <class R>
void swap(promise<R>& x, promise<R>& y) noexcept;
```